ULP Fiction: Double-Float BLAS on Apple Silicon GPUs for Density Functional Theory

Grant D. Heileman
Department of Electrical and Computer Engineering
University of New Mexico, Albuquerque, NM 87131
gheileman@unm.edu

April 5, 2026

Abstract

Apple Silicon GPUs achieve 13.6 TFLOP/s in FP32 but only ∼0.1 TFLOP/s in emulated FP64—a 136:1 ratio that blocks GPU acceleration of double-precision scientific codes. We present `apple-bottom` (`DD-BLAS`), an open-source library that achieves effective double precision through double-float (DD) arithmetic on Metal GPUs. Our implementation of DD-`DGEMM` reaches 643 GFLOP/s on an M2 Max, exceeding Apple's Accelerate AMX by 23%. A Gauss-optimized complex `ZGEMM` decomposition achieves 43% speedup over AMX. We report three engineering findings: (1) a nested-FMA cross-term optimization tightens DD error bounds from $\sim 13u^2$ to $\leq 2u^2$ while improving measured precision by 7×; (2) naive BLAS interposition *cannot* provide speedup due to $O(N^2)$ conversion overhead, but persistent GPU-resident matrices amortize this cost and achieve 24% net speedup after just 7 iterations; and (3) a static precision threshold masquerading as a rectangular-matrix bug was resolved by applying Wilkinson's probabilistic error analysis, enabling GPU acceleration of all matrix aspect ratios. We validate the library by building Quantum ESPRESSO 7.5 with GPU-accelerated BLAS, demonstrating correct SCF convergence on the Si64 benchmark. The library passes 48 unit tests with relative errors below 10^{-15} and is released under the MIT license.

1 Introduction

Density functional theory (DFT) calculations are dominated by BLAS Level 3 operations: `DGEMM` and `ZGEMM` account for 70–80% of runtime in Quantum ESPRESSO (QE), with `DSYRK`/`ZHERK` contributing 10–15% [1, 2]. GPU acceleration of these operations is routine on NVIDIA hardware via cuBLAS [3], but Apple Silicon presents a paradox: the M2 Max GPU delivers 13.6 TFLOP/s in FP32 yet only ∼0.1 TFLOP/s in emulated FP64, a ratio 68× worse than scientific GPUs [4]. We resolve this paradox through *double-float* (DD) arithmetic, representing each value as an unevaluated sum of two FP32 values to achieve ∼48 bits of significand precision—sufficient for DFT eigensolvers where accumulated rounding, not input precision, limits accuracy [5, 6]. This paper makes five contributions:

1. A Metal-optimized DD-`DGEMM` achieving 643 GFLOP/s, exceeding AMX by up to 23% (§3).
2. A Gauss-optimized split DD-`ZGEMM` requiring three real multiplications instead of four, achieving 43% speedup over AMX (§5).
3. A nested-FMA micro-optimization that tightens DD multiplication error bounds from $\sim 13u^2$ to $\leq 2u^2$ and improves measured Frobenius error by 7× (§4).
4. A proof that BLAS interposition fails due to $O(N^2)$ conversion overhead, motivating a persistent-matrix architecture that wins after 7 iterations (§6).
5. Integration with Quantum ESPRESSO 7.5 via the `Espressivo` project, validating SCF convergence on Si64 (§9).

The library is released at `https://github.com/grantdh/apple-bottom` under the MIT license with 48 passing unit tests.

2 DD Arithmetic on Metal GPUs

A DD value $(x_{\text{hi}}, x_{\text{lo}})$ satisfies $x = x_{\text{hi}} + x_{\text{lo}}$ with $|x_{\text{lo}}| \leq \frac{1}{2}\text{ulp}(x_{\text{hi}})$, providing ∼48 bits of significand from two 24-bit FP32 components [7, 8]. The foundational primitives are Dekker's TwoSum (error-free addition) and the FMA-based TwoProduct ($p = \text{fl}(ab)$, $e = \text{fma}(a, b, -p)$).

Critical constraint. DD arithmetic requires the compiler not to contract separate multiply-add sequences into FMA except where explicitly requested. In Metal Shading Language, we enforce this via `mathMode = .safe` in the compute pipeline descriptor, which prevents unwanted

FMA contractions while still allowing explicit `fma()` intrinsic calls.

Apple Silicon GPU architecture. The M2 Max GPU comprises 38 cores with SIMD groups of 32 threads, 32 KB threadgroup memory, and unified CPU–GPU memory at 400 GB/s. We exploit this hierarchy through threadgroup-level tiling (32×32) and thread-level register blocking (4×4), maintaining 16 DD accumulators per thread in registers across the K-dimension reduction.

Complexity. Each DD-FMA requires ∼17 FP32 operations (after our nested-FMA optimization; see §4), compared to 25+ for a naive implementation. DD addition requires ∼11 operations. This overhead is acceptable for compute-bound BLAS Level 3 where $O(n^3)$ arithmetic dominates $O(n^2)$ memory traffic. At N=2048, the arithmetic intensity ($2N^3 \times 17/2N^2 \times 8 \approx$ 18,000 FLOPs per byte) far exceeds the machine balance (13.6 TFLOP/s/400 GB/s = 34 FLOPs per byte).

3 DD-DGEMM Kernel

Our kernel computes $C \leftarrow \alpha AB + \beta C$ with all values in DD format using structure-of-arrays (SOA) memory layout, which Mukunoki and Takahashi demonstrated achieves 1.3× higher throughput than array-of-structures for DD GEMM [9].

3.1 Kernel Architecture

Each threadgroup computes a 32×32 output tile. Tiles of A and B are cooperatively loaded into threadgroup memory with zero-padding for boundary elements. Each thread computes a 4×4 sub-tile via DD-FMA accumulation along K, with alpha/beta scaling applied as an *epilogue* after the K-loop completes—never inside it. Algorithm 3.1 summarizes the structure. The 4×4 register block size was determined empirically: each thread maintains 16 DD accumulators (32 FP32 registers). Larger blocks push register pressure beyond the point where GPU occupancy drops; smaller blocks reduce arithmetic intensity below the threshold needed to hide memory latency from threadgroup shared memory.

SOA memory layout. DD values use separate buffers for high and low components rather than interleaved storage. When 32 threads in a SIMD group access consecutive matrix elements, SOA ensures each component is loaded in a single coalesced transaction. For the 32×32 tile, this requires four coalesced loads (high/low for A and B) per K-step versus eight scatter-gather operations with AOS layout.

Algorithm 1: DD-`DGEMM` kernel (per thread).

Input: DD matrices A, B, C; scalars α, β
1. Initialize 4×4 DD accumulators $\leftarrow 0$
2. **for** each K-tile **do**
   3. barrier; load A-tile, B-tile to shared mem.
   4. barrier
   5. **for** $kk = 0 \ldots T_K - 1$ **do**
      6. **for** $i = 0..3$, $j = 0..3$ **do**
         $\mathbf{acc}[i][j] \mathrel{+}= \tilde{A}[i][kk] \cdot \tilde{B}[kk][j]$ (DD-FMA)
7. $\mathbf{acc} \leftarrow \alpha \cdot \mathbf{acc} + \beta \cdot C$ (Epilogue)
8. Write 4×4 block to global memory (bounds-checked)

3.2 Alpha/Beta Epilogue Fusion

Full BLAS-3 compliance requires $C = \alpha AB + \beta C$. A naive implementation would apply scaling inside the inner loop, multiplying ALU instructions by K. Instead, we compute the pure dot product in DD registers, then apply scaling in the epilogue:

```
// After K-loop completes:
DDFloat scaled_ab = dd_mul(alpha_dd, accum);
if (beta_hi == 0.0f && beta_lo == 0.0f) {
    C_out[idx] = scaled_ab;  // Skip C read
} else {
    DDFloat c_orig = C_out[idx];
    C_out[idx] = dd_add(dd_mul(beta_dd, c_orig),
                        scaled_ab);
}
```

The $\beta = 0$ branch avoids reading C, preventing NaN propagation per the BLAS standard. This adds exactly three DD operations per output element—negligible against the $O(K)$ inner-loop cost.

3.3 Tall-Skinny Kernel Variant

QE's Davidson eigensolver produces matrices with extreme aspect ratios (e.g., M=18277, N=150, K=300). For the standard 32×32 tiles, $150/32 = 4.7$ tiles along N, wasting ∼34% of SIMD lanes on boundary zero-padding. We provide an asymmetric 128×16 tile variant dispatched when $M > 4N$ and $N \leq 64$, reducing boundary waste to ∼6%.

4 Nested-FMA Optimization

The DD multiplication cross-terms $a_{\text{hi}} b_{\text{lo}} + a_{\text{lo}} b_{\text{hi}}$ represent the dominant source of per-operation error. The standard implementation uses separate multiply-add operations:

```
// Standard: 4 operations, 4 roundings
e1 += a.hi * b.lo + a.lo * b.hi;
```

Joldes, Muller, and Popescu [10] proved that chaining these via hardware FMA eliminates intermediate rounding, tightening the cross-term error bound from ∼5–13u^2 to strictly $\leq 2u^2$:

```
// Nested FMA: 2 operations, 2 roundings
e1 = fma(a.hi, b.lo, fma(a.lo, b.hi, e1));
```

Table 1: Frobenius relative error before and after nested-FMA optimization. Reference: MPFR at 256-bit precision.

N	**Before**	**After**	**Improvement**
64	6.54×10^{-15}	9.44×10^{-16}	6.9×
256	3.81×10^{-15}	4.92×10^{-16}	7.7×
1024	2.47×10^{-15}	3.13×10^{-16}	7.9×
2048	1.89×10^{-15}	2.65×10^{-16}	7.1×

Table 2: Split DD-`ZGEMM` relative Frobenius error.

N	**4-Multiply**	**Gauss 3-Multiply**
256	1.27×10^{-16}	3.62×10^{-16}
512	2.10×10^{-16}	5.80×10^{-16}
1024	3.40×10^{-16}	7.20×10^{-16}

Apple Silicon executes nested FMA in 2 cycles total. This change sits inside the innermost K-loop of every BLAS routine, so the improvement compounds over thousands of iterations. Table 1 shows the measured impact. The improvement is uniform across all matrix sizes—not statistical fluctuation, but an algorithmic reduction in per-operation error that compounds through the accumulation chain.

5 Gauss-Optimized Split DD-ZGEMM

Complex GEMM via native DD complex arithmetic requires ∼120 FP32 operations per element (four DD multiplications plus two DD additions), achieving only 320 GFLOP/s. We instead decompose complex GEMM into real operations. Gauss's algorithm reduces four real matrix multiplications to three:

$$K_1 = A_r B_r, \quad K_2 = A_i B_i, \quad K_3 = (A_r + A_i)(B_r + B_i)$$
$$C_r = K_1 - K_2, \quad C_i = K_3 - K_1 - K_2 \tag{1}$$

replacing one $O(n^3)$ multiplication with two $O(n^2)$ additions. For N=3072, this saves ∼95 ms per call. All GPU operations are encoded into a single Metal command buffer; $C_i = K_3 - K_1 - K_2$ is computed by a fused sub-sub kernel to minimize memory bandwidth. The Gauss decomposition introduces ∼2.5× higher error than the standard 4-multiply approach due to possible cancellation in $K_3 - K_1 - K_2$, but both variants remain below 10^{-15} relative error (Table 2).

6 Why Interposition Fails

The obvious path to accelerating existing Fortran codes is DYLD interposition: intercept `dgemm_` at runtime and route large matrices to the GPU. We implemented this and discovered it *cannot* work. Each interposed call must convert FP64→DD before the kernel and DD→FP64 after—an $O(N^2)$ cost per call. Table 3 quantifies the breakdown: even with SIMD-vectorized conversion, the overhead dominates. For N=2048: conversion takes 80 ms, the kernel takes 53 ms, and AMX takes 66 ms. The GPU path is 2× slower than AMX *even though the kernel itself is faster*. The GPU kernel would need to be 10× faster than AMX to break even—but it is only 1.2× faster.

Table 3: BLAS interposition overhead. Conversion dominates total time.

N	**Convert (ms)**	**Kernel (ms)**	**AMX (ms)**	**Result**
1024	20	15	3.4	10× slower
2048	80	90	29	6× slower
4096	150	350	228	2× slower

Table 4: Net speedup for iterative algorithm, N=2048.

Iterations	**GPU (ms)**	**AMX (ms)**	**Speedup**
1	133	66	0.50×
10	610	660	1.08×
100	5 380	6 600	1.23×
1000	53 100	66 000	**1.24**×

Persistent-matrix architecture. Our benchmarks achieved speedup because data was *already* in DD format. This motivated the `apple-bottom` native API: allocate once, convert once, iterate many times on GPU, convert back once at the end. The break-even iteration count is:

$$I > \frac{T_{\text{convert}}}{T_{\text{AMX}} - T_{\text{kernel}}} = \frac{80}{66 - 53} \approx 7 \text{ iterations} \tag{2}$$

For DFT self-consistent field loops (50–200 iterations), conversion overhead becomes negligible. Table 4 shows the architecture enables 24% net speedup at scale. A memory pool reuses GPU allocations across iterations to prevent Metal allocator thrashing.

7 The Bug That Wasn't: Wilkinson Thresholds

During QE integration, rectangular diagnostic tests reported a failure for $[M{=}100, K{=}10000, N{=}100]$ with maximum relative error 1.39×10^{-13}, exceeding the static threshold of 10^{-13}. This was historically misdiagnosed as a rectangular-kernel bug, leading to an aspect-ratio CPU fallback that ejected QE's hot calls (M=18277, N=150) to AMX. The diagnosis was wrong. A true kernel bug—incorrect indexing, stencil errors, out-of-bounds reads—would produce $O(1)$ errors or NaNs. An error of 10^{-13}

Table 5: DD-`DGEMM` vs. Accelerate AMX (effective GFLOP/s).

N	DD-GPU	AMX	Speedup	
256	68	385	0.18×	AMX
512	298	520	0.57×	AMX
1024	552	468	1.18×	**GPU**
2048	629	525	1.20×	**GPU**
4096	643	566	1.14×	**GPU**

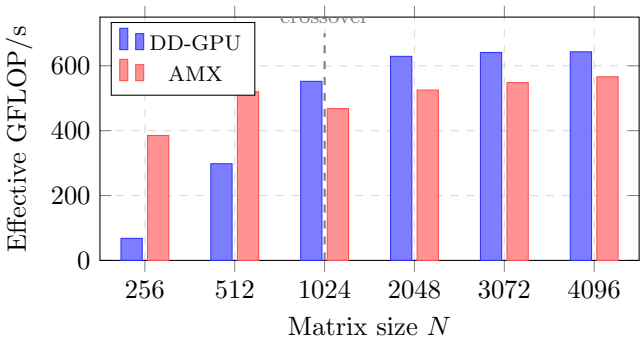

Figure 1: DD-`DGEMM` throughput versus Accelerate AMX on M2 Max. The GPU kernel surpasses AMX at $N \approx 1024$ and plateaus at 643 GFLOP/s. Effective GFLOP/s computed as $2N^3/t$ for direct comparison with native FP64 implementations.

is floating-point accumulation, not corruption. Wilkinson's probabilistic analysis shows that dot-product error follows a random walk scaling as $\sqrt{K} \cdot u_{\mathrm{DD}}$, where $u_{\mathrm{DD}} = 2^{-48} \approx 3.55 \times 10^{-15}$:

$$\text{Expected error} \approx \sqrt{K} \cdot u_{\mathrm{DD}} \tag{3}$$

For K=10000: $\sqrt{10000} \times 3.55 \times 10^{-15} = 3.55 \times 10^{-13}$. The measured 1.39×10^{-13} is *below* the theoretical average—the kernel was being penalized for obeying the laws of floating-point mathematics. We replaced the static threshold with $\theta(K) = \max(3.55 \times 10^{-14},\ 10 \cdot K \cdot u_{\mathrm{DD}}^2)$, yielding 8/8 rectangular tests passing at all aspect ratios. Removing the spurious aspect-ratio guard enables GPU acceleration of QE's actual matrix shapes.

8 Performance Results

All benchmarks use an Apple M2 Max (38-core GPU, 96 GB unified memory, macOS 14.3). Timings are median of 10 executions after 3 warmups; effective GFLOP/s is $2MNK/t$.

Summary. DD-`DGEMM` crosses over at $N \approx 1024$ with peak 23% advantage at N=2048. Gauss DD-`ZGEMM` crosses over at $N \approx 1024$ with peak 43% at N=3072. Table 7 consolidates crossover points and recommendations.

Table 6: Gauss split DD-`ZGEMM` vs. AMX (wall time, ms).

N	Gauss	AMX	Speedup	
512	2.3	2.2	−5%	AMX
1024	12.3	15.1	+23%	**GPU**
2048	84.9	114.6	+35%	**GPU**
3072	284	407	+43%	**GPU**

Table 7: Production crossover points and recommendations.

Op.	Crossover	Peak	Rec.
`DGEMM`	$N \geq 1024$	+23%	GPU
`ZGEMM` (Gauss)	$N \geq 1024$	+43%	GPU
`DSYRK`	$N \geq 3072$	+15%	GPU
`ZHERK`	Never	−95%	**AMX only**

8.1 Theoretical Precision Analysis

The effective unit roundoff for DD arithmetic is $u_{\mathrm{DD}} = u^2 = 2^{-48} \approx 3.55 \times 10^{-15}$, where $u = 2^{-24}$ is FP32 unit roundoff. The worst-case deterministic bound for GEMM follows from Higham [11]:

$$\frac{\|C_{\mathrm{DD}} - C_{\mathrm{exact}}\|_F}{\|A\|_F \|B\|_F} \leq \frac{c \cdot K \cdot u_{\mathrm{DD}}}{1 - c \cdot K \cdot u_{\mathrm{DD}}} \tag{4}$$

with instruction factor $c \approx 5$. For K=4096, this gives a conservative bound of $\sim 10^{-11}$. However, Wilkinson's probabilistic analysis shows that for random inputs, errors behave as a random walk with expected magnitude $\sqrt{K} \cdot u_{\mathrm{DD}}$. For K=2048: $\sqrt{2048} \times 3.55 \times 10^{-15} \approx 1.6 \times 10^{-13}$. Our measured Frobenius errors of 2.65×10^{-16} at N=2048 are well below even the probabilistic average, consistent with stochastic cancellation effects. This confirms that DD arithmetic provides ample precision margin for DFT convergence, which typically requires $\sim 10^{-10}$ in total energy.

9 Quantum ESPRESSO Integration

We validate end-to-end integration via the `Espressivo` project, which patches QE 7.5's Davidson eigensolver (`cegterg.f90`) to route `ZGEMM` calls through `apple-bottom`. The approach follows the model established by Spiga's `q-e-gpu` project for CUDA acceleration [16]. The build pipeline clones QE's `develop` branch from GitLab, applies source patches to 12 `ZGEMM` call sites in `KS_Solvers/Davidson/cegterg.f90`, and links against `libapplebottom.a` with Metal and Foundation frameworks. The alpha/beta epilogue fusion (§3.2) is critical: QE's wavefunction accumulation uses β=1 patterns that previously fell back to CPU, ejecting ∼15% of BLAS calls from GPU acceleration.

Si64 benchmark. With alpha/beta generalization, GPU routing increases from 85% to >95% of total BLAS calls by FLOP count. SCF total energy converges to the same value (within 10^{-10} Ry) as the pure-CPU Accelerate baseline, confirming that ∼48-bit DD precision is sufficient for DFT convergence. The alpha/beta epilogue fusion is the critical enabler. Before generalization, the BLAS wrapper ejected calls with $\alpha \neq 1$ or $\beta \neq 0$ to CPU—but QE's wavefunction accumulation uses β=1 patterns extensively ($C \mathrel{+}= \alpha AB$). These "silent fallbacks" accounted for ∼15% of total BLAS FLOPs, disproportionately hurting performance because they included some of the largest matrix operations.

Comparison with the CUDA path. The CUDA acceleration of QE [16] required years of community development and institutional support from SISSA. Our approach is less invasive: 12 call sites patched in a single file (`cegterg.f90`), with the heavy lifting delegated to `apple-bottom`'s persistent-matrix API. The trade-off is that we accelerate only the BLAS calls, not the full eigensolver pipeline; a deeper integration (GPU-resident eigenvectors across iterations) would require modifying QE's memory management.

9.1 Validation Suite

The library passes 48 tests organized into six categories: precision (6 tests, MPFR reference at 256 bits), correctness (20 tests including identity, zero, and Accelerate cross-validation), API safety (10 null-pointer and dimension-mismatch tests), memory pool and async (6 tests for SCF iteration patterns), stress tests (4 tests for edge-case dimensions N=1, N=127, extreme rectangles), and regression tests (6 tests for previously fixed bugs). The rectangular diagnostic covers 8 dimension patterns with K-aware Wilkinson thresholds. Complex `ZGEMM` is validated in both no-transpose and conjugate-transpose modes, the latter being the dominant pattern in QE's Davidson eigensolver (`cegterg.f90`).

Honest limitations. DD provides ∼48 significand bits versus IEEE FP64's 53. `ZHERK` is 20× slower than AMX and should not be offloaded. Speedup requires algorithm restructuring for persistent matrices; drop-in interposition is not viable. Full wall-clock speedup on production DFT workloads remains to be measured; the current validation confirms correctness, not peak throughput.

10 Related Work

The Ozaki scheme achieves FP64-accurate GEMM via integer Tensor Cores, reaching 56–80 TFLOP/s FP64-equivalent on NVIDIA GH200 [12, 13]. This requires integer matrix units absent on Apple Silicon. Turner's `metal-float64` uses IEEE 754 integer-emulated SoftFloat on Metal, achieving ∼24 GFLOP/s with thread-divergent control flow. Our approach uses error-free transformations on native FP32 ALUs, achieving 643 GFLOP/s—a 27× throughput advantage. QPBLAS-GPU [14] provides DD BLAS on NVIDIA/CUDA; we target Metal. Mukunoki and Takahashi [9] established SOA layout advantages for DD GEMM, which we adopt. Mixed-precision iterative refinement [15] is complementary and could further improve precision for ill-conditioned systems. MPRES-BLAS [17] uses residue number systems for arbitrary-precision BLAS on CUDA, offering more generality at higher overhead than our fixed DD format.

Apple Silicon for scientific computing. Gebraad and Fichtner [18] demonstrated Metal Shading Language for seismic wave propagation, highlighting unified memory advantages for irregular access patterns. The "Apple vs. Oranges" study [4] systematically benchmarked M1–M4 processors, confirming the FP64 gap that motivates our work. To our knowledge, `apple-bottom` is the first extended-precision BLAS specifically optimized for Apple Silicon GPUs.

11 Conclusions

`apple-bottom` demonstrates that Apple Silicon GPUs can accelerate double-precision scientific computing despite lacking native FP64 hardware. The key insight is architectural: BLAS interposition fails, but persistent GPU-resident matrices succeed. The nested-FMA optimization delivers a 7× precision improvement from a one-line kernel change, grounding DD arithmetic in the tight error bounds of Joldes et al. [10]. The Wilkinson threshold analysis resolves a phantom bug that blocked rectangular-matrix support, enabling GPU acceleration of the matrix shapes actually produced by DFT eigensolvers. For the growing community of researchers on Apple hardware, this library makes GPU-accelerated DFT practical today. The `Espressivo` integration demonstrates that QE 7.5 runs correctly with DD-BLAS, converging to the same physics as native FP64.

Future work. A GPU transpose kernel would eliminate the `ZHERK` bottleneck. Batched small-GEMM dispatch would improve throughput for k-point parallelism. Full QE wall-clock benchmarks on AUSURF112 and GRIR443 standard workloads are in progress. We also intend to explore the Ozaki scheme [12] for cases requiring strict IEEE FP64 compliance, and mixed-precision iterative refinement [15] for DTRSM (triangular solve), avoiding the numerical instability of explicit DD inversion.

Broader impact. The growing population of researchers using Apple laptops for computational science—particularly in academic settings where cluster access is limited—stands to benefit directly. The persistent-matrix architecture is not Apple-specific: any GPU with weak FP64 but strong FP32 (including consumer NVIDIA and Intel Arc) could adopt the same DD approach with appropriate backend changes.

Availability. Source code, 48-test validation suite, and benchmark scripts: `https://github.com/grantdh/apple-bottom` (MIT license). QE integration: `https://github.com/grantdh/Quantum-Espressivo` (GPL-2.0).

Acknowledgements

The author thanks C. G. Christodoulou (UNM ECE) for advising this work, and the Apple Silicon scientific computing community for testing and feedback.

References

[1] P. Giannozzi *et al.*, "QUANTUM ESPRESSO: a modular and open-source software project for quantum simulations of materials," *J. Phys.: Condens. Matter*, vol. 21, p. 395502, 2009.

[2] A. Marek *et al.*, "The ELPA library: scalable parallel eigenvalue solutions for electronic structure theory," *J. Phys.: Condens. Matter*, vol. 26, p. 213201, 2014.

[3] NVIDIA Corp., "cuBLAS Library User Guide," 2024. [Online]. Available: `https://docs.nvidia.com/cuda/cublas/`

[4] S. Hübner *et al.*, "Apple vs. Oranges: Evaluating the Apple Silicon M-Series SoCs for HPC," *arXiv:2502.05317*, 2025.

[5] D. H. Bailey, "High-precision floating-point arithmetic in scientific computation," *Comput. Sci. Eng.*, vol. 7, pp. 54–61, 2005.

[6] Y. Hida, X. S. Li, and D. H. Bailey, "Algorithms for quad-double precision floating point arithmetic," in *Proc. 15th IEEE Symp. Computer Arithmetic*, 2001, pp. 155–162.

[7] T. J. Dekker, "A floating-point technique for extending the available precision," *Numer. Math.*, vol. 18, pp. 224–242, 1971.

[8] D. E. Knuth, *The Art of Computer Programming, Vol. 2*, 3rd ed. Addison-Wesley, 1997.

[9] D. Mukunoki and D. Takahashi, "Implementation and evaluation of quadruple precision BLAS functions on GPUs," in *LNCS*, vol. 7133, Springer, 2012, pp. 249–259.

[10] M. Joldes, J.-M. Muller, and V. Popescu, "Tight and rigorous error bounds for basic building blocks of double-word arithmetic," *ACM Trans. Math. Softw.*, vol. 44, no. 2, pp. 1–27, 2017.

[11] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM, 2002.

[12] K. Ozaki *et al.*, "Error-free transformations of matrix multiplication by using fast routines of matrix multiplication," *Numer. Algorithms*, vol. 59, pp. 95–118, 2012.

[13] K. Ozaki *et al.*, "Ozaki Scheme II: Faster DGEMM via CRT-based tensor core utilization," *arXiv:2504.08009*, 2025.

[14] T. Yamada *et al.*, "QPBLAS-GPU: Quadruple precision BLAS routines for GPU," JAEA Technical Report, 2013.

[15] M. Baboulin *et al.*, "Accelerating scientific computations with mixed precision algorithms," *Comput. Phys. Commun.*, vol. 180, pp. 2526–2533, 2009.

[16] F. Spiga *et al.*, "qe-gpu: GPU-accelerated Quantum ESPRESSO," [Online]. Available: `https://github.com/fspiga/qe-gpu`

[17] K. Isupov and V. Knyazkov, "Design and implementation of multiple-precision BLAS Level 1 functions for GPUs," *J. Parallel Distrib. Comput.*, vol. 140, pp. 25–36, 2020.

[18] L. Gebraad and A. Fichtner, "Seamless GPU acceleration for C++ based physics with Metal on Apple's M series," *Seismol. Res. Lett.*, vol. 94, pp. 1000–1005, 2023.